



Smart Contract Audit Report

for

NeutronX

06-2026

Contents

1 Introduction	3
1.1 Project Summary	3
1.2 Vulnerability Summary	3
2 Findings	4
3 Detailed Results	5
3.1 cross-chain-slots-can-be-grief-locked-cheaply-because-fill-credentials-are-public-while-the-safety-bond-is-flat	5
3.2 non-standard-output-tokens-can-satisfy-nominal-accounting-while-underpaying-the-swapper	5
3.3 settlement-critical-watchers-can-silently-skip-historical-cross-chain-events-after-restarts-or-long-outages	6
3.4 cross-chain-nonce-is-documented-as-anti-replay-but-is-never-enforced-so-multiple-same-nonce-orders-can-remain-live-simultaneously	6
3.5 same-chain-collateral-can-still-be-manipulated-materially-because-it-relies-on-a-short-single-pool-twap-with-no-liquidity-sanity-checks	7
3.6 reopened-cross-chain-slots-cannot-be-retried-by-the-same-filler-address-on-the-destination-chain	7
3.7 fillauction-can-trap-eth-refunds-and-treasury-proceeds-for-non-payable-recipients	8
3.8 same-chain-fallback-support-is-narrower-than-order-creation-and-reactor-settlement-so-unsupported-tokens-silently-lose-the-fallback-safety-net	8
4 Appendix	9
4.1 Severity Definitions	9

1 Introduction

Trufy has been engaged by what to perform a security audit of the NeutronX smart contracts. The purpose of this audit is to achieve the followings:

- Ensure that smart contract functions work as intended.
- Identify possible vulnerabilities, which could be exploited by an attacker.
- Identify smart contract bugs, which might lead to unexpected behavior.
- Make recommendations to improve code safety and readability. As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage.

1.1 Project Summary

- Project Name: NeutronX
- Audit method: codex

1.2 Vulnerability Summary

Severity	# of Findings
High	3
Medium	4
Low	1
Informational	0
Optimization	0

2 Findings

The security audit identified 8 findings across different severity levels. Below is a summary of all identified issues:

1. cross-chain-slots-can-be-grief-locked-cheaply-because-fill-credentials-are-public-while-the-safety-bond-is-flat

Severity: high

Affected Elements: 3

2. non-standard-output-tokens-can-satisfy-nominal-accounting-while-underpaying-the-swapper

Severity: high

Affected Elements: 2

3. settlement-critical-watchers-can-silently-skip-historical-cross-chain-events-after-restarts-or-long-outages

Severity: high

Affected Elements: 2

4. cross-chain-nonce-is-documented-as-anti-replay-but-is-never-enforced-so-multiple-same-nonce-orders-can-remain-live-simultaneously

Severity: medium

Affected Elements: 3

5. same-chain-collateral-can-still-be-manipulated-materially-because-it-relies-on-a-short-single-pool-twap-with-no-liquidity-sanity-checks

Severity: medium

Affected Elements: 2

6. reopened-cross-chain-slots-cannot-be-retried-by-the-same-filler-address-on-the-destination-chain

Severity: medium

Affected Elements: 2

7. fillauction-can-trap-eth-refunds-and-treasury-proceeds-for-non-payable-recipients

Severity: medium

Affected Elements: 1

8. same-chain-fallback-support-is-narrower-than-order-creation-and-reactor-settlement-so-unsupported-tokens-silently-lose-the-fallback-safety-net

Severity: low

Affected Elements: 2

3 Detailed Results

3.1 cross-chain-slots-can-be-grief-locked-cheaply-because-fill-credentials-are-public-while-the-safety-bond-is-flat

Severity: high

Affected Code Elements (3)

node uint256 public constant MIN_SAFETY_DEPOSIT = 0.001 ether; contract/src/crosschain/EscrowSrcFactory.sol

Source Mapping

File: contract/src/crosschain/EscrowSrcFactory.sol

Lines: 148-148

Columns: -

Start Byte:

Length: bytes

Code

contract/src/crosschain/EscrowSrcFactory.sol

```
148 uint256 public constant MIN_SAFETY_DEPOSIT = 0.001 ether;
```

node EscrowSrc(escrow).initialize(value: msg.value)(hashlock, msg.sender, order.swapper, order.inputToken, amount, order.deadline); contract/src/crosschain/EscrowSrcFactory.sol

Source Mapping

File: contract/src/crosschain/EscrowSrcFactory.sol

Lines: 302-304

Columns: -

Start Byte:

Length: bytes

Code

contract/src/crosschain/EscrowSrcFactory.sol

```
302 EscrowSrc(escrow).initialize{value: msg.value}(
    hashlock, msg.sender, order.swapper, order.inputToken, amount,
304 order.deadline
304 );
```

node slots: slots.rows.map(s => ({ index: s.slot_index, hashlock: s.hashlock, proof: JSON.parse(s.proof) as string[], status: s.status, assignedFiller: s.assigned_filler, escrowAddr: s.escrow_addr, })) backend/src/services/crosschainService.ts

Source Mapping

File: backend/src/services/crosschainService.ts

Lines: 462-469

Columns: -

Start Byte:

Length: bytes

Code

backend/src/services/crosschainService.ts

```
462 slots: slots.rows.map(s => ({
463     index:      s.slot_index,
464     hashlock:   s.hashlock,
465     proof:      JSON.parse(s.proof) as string[],
466     status:     s.status,
467     assignedFiller: s.assigned_filler,
468     escrowAddr:  s.escrow_addr,
```

Description

`fillSlot()` is permissionless, binds the caller as the filler, and only requires a flat `0.001 ether` bond. The backend also returns `cosignerSig`, `swapperSig`, `hashlock`, and `proof` in the public order payload, so any observer can front-run a slot fill, lock the swapper's source assets into a fresh escrow, and abandon the destination leg until expiry.

Recommendation

Gate `fillSlot()` with an on-chain filler authorization tied to the slot or signed assignment, and scale the safety deposit with slot value or minimum output so griefing cannot be done for a near-constant cost. Avoid exposing claim credentials publicly before the intended filler is committed.

3.2 non-standard-output-tokens-can-satisfy-nominal-accounting-while-underpaying-the-swapper

Severity: high

Affected Code Elements (2)

node uint256 paid = _paidOutput[orderHash] + outputAmount;
_paidOutput[orderHash] = paid; contract/src/PartialFillReactor.sol

Source Mapping

File: contract/src/PartialFillReactor.sol

Lines: 173-174

Columns: -

Start Byte:

Length: bytes

Code

contract/src/PartialFillReactor.sol

```
173 uint256 paid = _paidOutput[orderHash] + outputAmount;  
174     _paidOutput[orderHash] = paid;
```

node IERC20(order.info.outputToken).safeTransferFrom(msg.sender,
order.info.swapper, outputAmount); contract/src/PartialFillReactor.sol

Source Mapping

File: contract/src/PartialFillReactor.sol

Lines: 180-180

Columns: -

Start Byte:

Length: bytes

Code

contract/src/PartialFillReactor.sol

```
180 IERC20(order.info.outputToken).safeTransferFrom(msg.sender,  
order.info.swapper, outputAmount);
```

Description

`executePartialChunk()` increments `\_paidOutput` using the nominal `outputAmount` before transferring the output token and never measures how many tokens the swapper actually receives. Fee-on-transfer or balance-distorting tokens can therefore satisfy reactor accounting while crediting the recipient less than the nominal amount.

Recommendation

Measure the swapper's output-token balance delta around the transfer and account for the actual received amount, or reject non-standard output tokens explicitly. Apply the same received-amount invariant to the cross-chain payout path.

3.3 settlement-critical-watchers-can-silently-skip-historical-cross-chain-events-after-restarts-or-long-outages

Severity: high

Affected Code Elements (2)

function **getCheckpoint**

backend/src/db/checkpoint.ts

Source Mapping

File: backend/src/db/checkpoint.ts

Lines: 15-20

Columns: -

Start Byte:

Length: bytes

Code

backend/src/db/checkpoint.ts

```
15 export async function getCheckpoint(name: string, currentBlock: number):  
    Promise<number> {  
20     const { rows } = await db.query('SELECT last_block FROM indexer_state WHERE  
        name = $1', [name])  
17     if (rows.length > 0) return Number(rows[0].last_block)  
        await db.query('INSERT INTO indexer_state (name, last_block) VALUES ($1,  
18         $2)', [name, currentBlock])  
19     return currentBlock  
20 }
```

function **resolveCheckpoint**

backend/src/db/checkpoint.ts

Source Mapping

File: backend/src/db/checkpoint.ts

Lines: 39-47

Columns: -

Start Byte:

Length: bytes

Code

backend/src/db/checkpoint.ts

```
39 export async function resolveCheckpoint(name: string, currentBlock: number,  
    label: string): Promise<number> {  
47     let last = await getCheckpoint(name, currentBlock)  
41     if (Math.abs(currentBlock - last) > MAX_CHECKPOINT_LAG) {  
42         console.warn(`${label}: checkpoint ${last} too far from tip  
        ${currentBlock} (chain reset?) - rewinding`)  
43         last = currentBlock  
44         await setCheckpoint(name, last)  
45     }  
46     return last  
47 }
```

Description

The checkpoint layer initializes unseen watchers at the current tip and also rewinds any lag above `1000` blocks directly to the current block. If the backend restarts after downtime or loses DB state, older `SlotFilled` or destination events are never replayed, which can leave `assigned\_filler` unset and cause secret reveal logic to refuse otherwise valid settlement.

Recommendation

Persist checkpoints conservatively and always replay missed ranges instead of seeding or rewinding to tip. If a safety cap is needed, page through historical logs in bounded chunks rather than discarding the backlog.

3.4 cross-chain-nonce-is-documented-as-anti-replay-but-is-never-enforced-so-multiple-same-nonce-orders-can-remain-live-simultaneously

Severity: medium

Affected Code Elements (3)

variable `_orders`

contract/src/crosschain/EscrowSrcFactory.sol

Source Mapping

File: contract/src/crosschain/EscrowSrcFactory.sol

Lines: 156-156

Columns: -

Start Byte:

Length: bytes

Code

contract/src/crosschain/EscrowSrcFactory.sol

```
156 mapping(bytes32 => OrderState) private _orders;
```

node

```
await db.query(` CREATE TABLE IF NOT EXISTS cc_orders (
  order_hash VARCHAR(66) PRIMARY KEY, swapper
  VARCHAR(42) NOT NULL, input_token VARCHAR(42) NOT
  NULL, input_amount VARCHAR(78) NOT NULL,
  output_token VARCHAR(42) NOT NULL, min_output
  VARCHAR(78) NOT NULL, deadline INTEGER NOT NULL,
  nonce VARCHAR(78) NOT NULL, num_slots SMALLINT NOT
  NULL, merkle_root VARCHAR(66) NOT NULL, cosigner_sig
  TEXT NOT NULL, swapper_sig TEXT, t2_expiry INTEGER
  NOT NULL, reactor_addr VARCHAR(42) NOT NULL,
  chain_a_id INTEGER NOT NULL, created_at TIMESTAMP
  DEFAULT NOW() ) `)
```

backend/src/services/crosschainService.ts

Source Mapping

File: backend/src/services/crosschainService.ts

Lines: 16-35

Columns: -

Start Byte:

Length: bytes

Code

backend/src/services/crosschainService.ts

```
16 await db.query(`
35   CREATE TABLE IF NOT EXISTS cc_orders (
18     order_hash  VARCHAR(66) PRIMARY KEY,
19     swapper     VARCHAR(42) NOT NULL,
20     input_token VARCHAR(42) NOT NULL,
21     input_amount VARCHAR(78) NOT NULL,
22     output_token VARCHAR(42) NOT NULL,
23     min_output  VARCHAR(78) NOT NULL,
24     deadline    INTEGER NOT NULL,
25     nonce       VARCHAR(78) NOT NULL,
26     num_slots   SMALLINT NOT NULL,
27     merkle_root VARCHAR(66) NOT NULL,
28     cosigner_sig TEXT NOT NULL,
29     swapper_sig TEXT,
30     t2_expiry   INTEGER NOT NULL,
31     reactor_addr VARCHAR(42) NOT NULL,
32     chain_a_id  INTEGER NOT NULL,
33     created_at  TIMESTAMP DEFAULT NOW()
```

```
34    )
35    `)`
```

```
await db.query(` INSERT INTO cc_orders (order_hash,
swapper, input_token, input_amount, output_token,
min_output, deadline, nonce, num_slots, merkle_root,
cosigner_sig, t2_expiry, reactor_addr, chain_a_id,
dst_chain_id) VALUES
node ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15) ON
CONFLICT (order_hash) DO NOTHING `, [structHash,
p.swapper, p.inputToken, p.inputAmount, p.outputToken,
p.minOutput, p.deadline, p.nonce, numSlots, merkleRoot,
cosignerSig, t2Expiry, reactorAddr, p.chainAId,
p.dstChainId])
```

backend/src/services/crosschainService.ts

Source Mapping

File: backend/src/services/crosschainService.ts

Lines: 351-358

Columns: -

Start Byte:

Length: bytes

Code

backend/src/services/crosschainService.ts

```
351 await db.query(`
352     INSERT INTO cc_orders
353     (order_hash, swapper, input_token, input_amount, output_token,
354     min_output,
355     deadline, nonce, num_slots, merkle_root, cosigner_sig, t2_expiry,
356     reactor_addr, chain_a_id, dst_chain_id)
357     VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15)
358     ON CONFLICT (order_hash) DO NOTHING
359     `, [structHash, p.swapper, p.inputToken, p.inputAmount, p.outputToken,
360     p.minOutput,
361     p.deadline, p.nonce, numSlots, merkleRoot, cosignerSig, t2Expiry,
362     reactorAddr, p.chainAId, p.dstChainId])
```

Description

The cross-chain order struct documents `nonce` as anti-replay, but the factory stores state only by `orderHash`, not by `(swapper, nonce)`. The backend schema likewise keys persisted orders by `order\_hash` only and inserts duplicates with the same nonce as long as the signed terms differ enough to change the hash. As a result, multiple same-nonce orders can coexist and be filled independently.

Recommendation

Track and consume `(swapper, nonce)` on-chain before accepting a first fill, and enforce a uniqueness constraint on `(swapper, nonce)` in the backend so a nonce cannot remain live across multiple orders.

3.5 same-chain-collateral-can-still-be-manipulated-materially-because-it-relies-on-a-short-single-pool-twap-with-no-liquidity-sanity-checks

Severity: medium

Affected Code Elements (2)

function toEthNotional

contract/src/libs/DynamicStakeLib.sol

Source Mapping

File: contract/src/libs/DynamicStakeLib.sol

Lines: N/A

Columns: -

Start Byte:

Length: bytes

Code

contract/src/libs/DynamicStakeLib.sol

```
1 function toEthNotional(
2     uint256 fillAmount,
3     address inputToken,
4     uint24 feeTier,
5     address weth,
6     address uniV3Factory,
7     uint32 twapWindow
8 ) internal view returns (uint256) {
9     if (uniV3Factory == address(0) || weth == address(0) || inputToken ==
weth) {
10         return fillAmount;
11     }
12     address pool = IUniswapV3Factory(uniV3Factory).getPool(inputToken,
weth, feeTier);
13     require(pool != address(0), "no twap pool");
14
15     uint32[] memory secondsAgos = new uint32[](2);
16     secondsAgos[0] = twapWindow;
17     secondsAgos[1] = 0;
18     (int56[] memory tickCumulatives, ) =
IUniswapV3Pool0racle(pool).observe(secondsAgos);
19
20     int56 delta = tickCumulatives[1] - tickCumulatives[0];
21     int24 meanTick = int24(delta / int56(uint56(twapWindow)));
22     if (delta < 0 && (delta % int56(uint56(twapWindow)) != 0)) meanTick-
-;
23
24     uint160 sqrtPriceX96 = TickMath.getSqrtPriceAtTick(meanTick);
25     return _quoteWeth(sqrtPriceX96, fillAmount, inputToken, weth);
26 }
```

node uint32 constant TWAP_WINDOW = 60;

contract/script/Deploy.s.sol

Source Mapping

File: contract/script/Deploy.s.sol

Lines: 25-25

Columns: -

Start Byte:

Length: bytes

Code

contract/script/Deploy.s.sol

```
25  uint32  constant  TWAP_WINDOW      = 60;
```

Description

Collateral sizing reads a single Uniswap V3 pool chosen by `(inputToken, WETH, feeTier)`, computes a mean tick over the configured lookback, and converts that directly into ETH notional with no liquidity floor, secondary oracle, or volatility sanity check. The deployment script configures only a 60-second window, so a manipulator who can distort that one pool long enough can materially underprice the required bond.

Recommendation

Use a longer observation window plus liquidity and deviation guards, or cross-check against an independent oracle before accepting the TWAP. Also set a non-zero `minCollateral` during deployment so manipulated quotes cannot drive stake close to zero.

3.6 reopened-cross-chain-slots-cannot-be-retried-by-the-same-filler-address-on-the-destination-chain

Severity: medium

Affected Code Elements (2)

node

bytes32 salt = _salt(orderHash, slotIndex, _attempt[orderHash][slotIndex]);

contract/src/crosschain/EscrowSrcFactory.sol

Source Mapping

File: contract/src/crosschain/EscrowSrcFactory.sol
Lines: 294-294 Columns: - Start Byte: Length: bytes

Code

contract/src/crosschain/EscrowSrcFactory.sol
294 bytes32 salt = _salt(orderHash, slotIndex, _attempt[orderHash][slotIndex]);

node

bytes32 salt = keccak256(abi.encodePacked(hashlock, msg.sender));

contract/src/crosschain/EscrowDstFactory.sol

Source Mapping

File: contract/src/crosschain/EscrowDstFactory.sol
Lines: 66-66 Columns: - Start Byte: Length: bytes

Code

contract/src/crosschain/EscrowDstFactory.sol
66 bytes32 salt = keccak256(abi.encodePacked(hashlock, msg.sender));

Description

Source-side retries increment an `attempt` counter and derive a fresh source escrow salt, but the destination factory derives its CREATE2 salt only from `hashlock` and the filler address. Because the hashlock is reused across attempts, the same filler always collides with the previously deployed destination escrow address and cannot retry with the same wallet.

Recommendation

Include a retry dimension such as the source attempt counter or a backend-signed retry nonce in the destination CREATE2 salt, and propagate that retry identifier through watcher validation so a reopened slot can be redeployed by the same filler address.

3.7 fillauction-can-trap-eth-refunds-and-treasury-proceeds-for-non-payable-recipients

Severity: medium

Affected Code Elements (1)

function

withdraw

contract/src/FillAuction.sol

Source Mapping

File: contract/src/FillAuction.sol

Lines: 277-283Columns: -Start Byte:Length: bytes

Code

contract/src/FillAuction.sol

```
277 function withdraw() external nonReentrant {
283     uint256 amount = pendingReturns[msg.sender];
279     require(amount > 0, "nothing to withdraw");
280     pendingReturns[msg.sender] = 0;
281     (bool ok,) = msg.sender.call{value: amount}("");
282     require(ok, "transfer failed");
283 }
```

Description

`FillAuction` accrues ETH liabilities in `pendingReturns`, but `withdraw()` always sends funds directly to `msg.sender` and offers no alternate recipient or fallback crediting path. If the owed address is a non-payable contract, every withdrawal attempt reverts and the ETH remains practically unrecoverable for that recipient.

Recommendation

Replace the direct push-only withdrawal with a pull-payment function that accepts a payable recipient parameter, or credit failed sends into a secondary claimable balance that can be withdrawn to another address.

3.8 same-chain-fallback-support-is-narrower-than-order-creation-and-reactor-settlement-so-unsupported-tokens-silently-lose-the-fallback-safety-net

Severity: low

Affected Code Elements (2)

variable **TOKEN_META**

backend/src/watcher/fallbackWatcher.ts

Source Mapping

File: backend/src/watcher/fallbackWatcher.ts

Lines: 45-52

Columns: -

Start Byte:

Length: bytes

Code

backend/src/watcher/fallbackWatcher.ts

```
45 const TOKEN_META: Record<string, { decimals: number; symbol: string }> = {  
52   '0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2': { decimals: 18, symbol:  
    'WETH' },  
47   '0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48': { decimals: 6, symbol:  
    'USDC' },  
48   '0xdAC17F958D2ee523a2206206994597C13D831ec7': { decimals: 6, symbol:  
    'USDT' },  
49   '0x6B175474E89094C44Da98b954EedeAC495271d0F': { decimals: 18, symbol: 'DAI'  
    },  
50   '0x2260FAC5E5542a773Aa44fBCfE7C193bc2C599': { decimals: 8, symbol:  
    'WBTC' },  
51   '0x514910771AF9Ca656af840dfff83E8264EcF986CA': { decimals: 18, symbol:  
    'LINK' },  
52 }
```

node **if (!tokenIn || !tokenOut) { console.warn('Unknown token: \${order.input_token} / \${order.output_token}') continue }**

backend/src/watcher/fallbackWatcher.ts

Source Mapping

File: backend/src/watcher/fallbackWatcher.ts

Lines: 209-212

Columns: -

Start Byte:

Length: bytes

Code

backend/src/watcher/fallbackWatcher.ts

```
209 if (!tokenIn || !tokenOut) {  
212     console.warn('Unknown token: ${order.input_token} /  
    ${order.output_token}')  
211     continue  
212 }
```

Description

Same-chain orders and reactor settlement are generic over ERC-20s, but the fallback watcher hardcodes metadata for only six tokens and aborts when either side of a pair is absent. Orders for other tokens can still exist and be partially filled, but they lose the fallback completion path near expiry because the watcher cannot quote or execute them.

Recommendation

Source token metadata dynamically from the order/token registry instead of a fixed map, or reject unsupported tokens at order-creation time so the system does not advertise fallback support it cannot execute.

4 Appendix

4.1 Severity Definitions

High

This level vulnerabilities are significant security issues that could potentially be exploited to cause considerable damage. They require immediate attention and should be fixed as soon as possible.

Medium

This level vulnerabilities are hard to exploit but very important to fix, they carry an elevated risk of smart contract manipulation, which can lead to critical-risk severity.

Low

This level vulnerabilities should be fixed, as they carry an inherent risk of future exploits, and hacks which may or may not impact the smart contract execution.

Informational

This level vulnerabilities can be ignored. They are code style violations and informational statements in the code. They may not affect the smart contract execution.

Optimization

Gas Optimization findings refer to exhibits that do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction on the total gas cost of a transaction.